

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Import of necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
```

1. Loading and merging the datasets

```
# Allocation of the file path to load the tourism rating dataset
toursim_rating_filepath="/content/drive/MyDrive/Abhijit/Capstone/Capstone_Heritage/Part2/tourism_rating.csv"
toursim_rating=pd.read_csv(toursim_rating_filepath)
toursim_rating.head()
```

	User_Id	Place_Id	Place_Ratings
0	1	179	3
1	1	344	2
2	1	5	5
3	1	373	3
4	1	101	4

Next steps:

[Generate code with tourism_rating](#)
[New interactive sheet](#)

```
toursim_rating.info() # information of the data
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   User_Id         10000 non-null  int64
1   Place_Id        10000 non-null  int64
2   Place_Ratings   10000 non-null  int64
dtypes: int64(3)
memory usage: 234.5 KB
```

```
# Allocation of the file path to load the tourism with index dataset
```

```
toursim_id_filepath="/content/drive/MyDrive/Abhijit/Capstone/Capstone_Heritage/Part2/tourism_with_id.xlsx"
tourism_with_id=pd.read_excel(toursim_id_filepath)
tourism_with_id.head(20)
```

	Place_Id	Place_Name	Description	Category	City	Price	Rating	Time_Minutes	Coordinate	Lat
0	1	Monumen Nasional	Monumen Nasional atau yang populer disingkat d...	Budaya	Jakarta	20000	4.6	15.0	{'lat': -6.1753924, 'lng': 106.8271528}	-6.175392 106.8271528
1	2	Kota Tua	Kota tua di Jakarta, yang juga bernama Kota Tu...	Budaya	Jakarta	0	4.6	90.0	{'lat': -6.137644799999999, 'lng': 106.8171245}	-6.137645 106.8171245
2	3	Dunia Fantasi	Dunia Fantasi atau disebut juga Dufan adalah t...	Taman Hiburan	Jakarta	270000	4.6	360.0	{'lat': -6.125312399999999, 'lng': 106.8335377}	-6.125312 106.8335377
3	4	Taman Mini Indonesia Indah (TMII)	Taman Mini Indonesia Indah merupakan suatu kaw...	Taman Hiburan	Jakarta	10000	4.5	NaN	{'lat': -6.302445899999999, 'lng': 106.8951559}	-6.302446 106.8951559
4	5	Atlantis Water Adventure	Atlantis Water Adventure atau dikenal dengan A...	Taman Hiburan	Jakarta	94000	4.5	60.0	{'lat': -6.12419, 'lng': 106.839134}	-6.124190 106.839134
5	6	Taman Impian Jaya Ancol	Taman Impian Jaya Ancol merupakan sebuah objek	Taman Hiburan	Jakarta	25000	4.5	10.0	{'lat': -6.117333200000001, 'lng': 106.8579951}	-6.117333 106.8579951

Next steps: [Generate code with tourism_with_id](#) [New interactive sheet](#)

tourism_with_id.info() # getting the information of the data

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 437 entries, 0 to 436
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Place_Id        437 non-null   int64
1   Place_Name      437 non-null   object
2   Description     437 non-null   object
3   Category       437 non-null   object
4   City            437 non-null   object
5   Price           437 non-null   int64
6   Rating          437 non-null   float64
7   Time_Minutes   205 non-null   float64
8   Coordinate      437 non-null   object
9   Lat             437 non-null   float64
10  Long            437 non-null   float64
11  Unnamed: 11     0 non-null     float64
12  Unnamed: 12     437 non-null   int64
dtypes: float64(5), int64(3), object(5)
memory usage: 44.5+ KB
```

tourism_with_id.columns # Check the present columns in the dataset

```
Index(['Place_Id', 'Place_Name', 'Description', 'Category', 'City', 'Price',
      'Rating', 'Time_Minutes', 'Coordinate', 'Lat', 'Long', 'Unnamed: 11',
      'Unnamed: 12'],
      dtype='object')
```

tourism_with_id[['Unnamed: 11', 'Unnamed: 12']]

	Unnamed: 11	Unnamed: 12
0	NaN	1
1	NaN	2
2	NaN	3
3	NaN	4
4	NaN	5
...	...	...
432	NaN	433
433	NaN	434
434	NaN	435
435	NaN	436
436	NaN	437

437 rows × 2 columns

These two columns are not necessary as they contains no values or place ID duplicate values

```
tourism_with_id.drop(['Unnamed: 11','Unnamed: 12'],axis=1,inplace=True) # dropping the unnecessary columns
tourism_with_id.head()
```

	Place_Id	Place_Name	Description	Category	City	Price	Rating	Time_Minutes	Coordinate	Lat	Lon
0	1	Monumen Nasional	Monumen Nasional atau yang populer disingkat d...	Budaya	Jakarta	20000	4.6	15.0	{'lat': -6.1753924, 'lng': 106.8271528}	-6.175392	106.82715
1	2	Kota Tua	Kota tua di Jakarta, yang iuaa	Budava	Jakarta	0	4.6	90.0	{'lat': -6.137644799999999, 'lng': 106.81712}	-6.137645	106.81712

Next steps: [Generate code with tourism_with_id](#) [New interactive sheet](#)

```
# Allocation of the file path to load the user dataset
```

```
user_filepath="/content/drive/MyDrive/Abhijit/Capstone/Capstone_Heritage/Part2/user.csv"
user=pd.read_csv(user_filepath)
user.head()
```

	User_Id	Location	Age
0	1	Semarang, Jawa Tengah	20
1	2	Bekasi, Jawa Barat	21
2	3	Cirebon, Jawa Barat	23
3	4	Bekasi, Jawa Barat	21
4	5	Lampung, Sumatera Selatan	20

Next steps: [Generate code with user](#) [New interactive sheet](#)

```
# Merge tourism_with_id_df into tourism_rating_df based on 'Place_Id'
merged_df = pd.merge(tourism_rating, tourism_with_id, on='Place_Id', how='left') # through left join because I want tourism_rating

# Check the result
merged_df.head()
```

	User_Id	Place_Id	Place_Ratings	Place_Name	Description	Category	City	Price	Rating	Time_Minutes	Coordinate
0	1	179	3	Candi Ratu Boko	Situs Ratu Baka atau Candi Boko (Hanacaraka:ꦏꦢꦶꦫꦠꦸꦧꦺꦴꦏ)	Budaya	Yogyakarta	75000	4.6	90.0	{'lat': -7.7926, 'lng': 110.3695}
1	1	344	2	Pantai Marina	Pantai Marina (bahasa Jawa: ꦥꦺꦤ꧀ꦠꦶꦩꦤꦶ)	Bahari	Semarang	3000	4.1	NaN	{'lat': -6.9931, 'lng': 110.4309}
				Atlantis	Atlantis Water	Taman					{'lat': -6.9931, 'lng': 110.4309}

Next steps: [Generate code with merged_df](#) [New interactive sheet](#)

```
merged_df['Place_Id'].value_counts() # check the unique length of the unique place_Id wheather matches with the loaded data
```

	count
Place_Id	
298	39
177	39
437	38
201	37
208	36
...	...
72	13
161	13
124	13
51	12
175	12

437 rows × 1 columns

dtype: int64

```
merged_df.info() # information of the merged dataset
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   User_Id         10000 non-null  int64
1   Place_Id        10000 non-null  int64
2   Place_Ratings   10000 non-null  int64
3   Place_Name      10000 non-null  object
4   Description     10000 non-null  object
5   Category        10000 non-null  object
6   City            10000 non-null  object
7   Price           10000 non-null  int64
8   Rating          10000 non-null  float64
9   Time_Minutes    4628 non-null   float64
10  Coordinate       10000 non-null  object
11  Lat              10000 non-null  float64
12  Long             10000 non-null  float64
dtypes: float64(4), int64(4), object(5)
memory usage: 1015.8+ KB
```

```
user.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   User_Id     300 non-null    int64
1   Location    300 non-null    object
2   Age         300 non-null    int64
dtypes: int64(2), object(1)
memory usage: 7.2+ KB
```

```
# Merge user_details_df into merged_df based on 'user_id'
final_merged_df = pd.merge(merged_df, user, on='User_Id', how='left')

# Check the result
final_merged_df.head()
```

	User_Id	Place_Id	Place_Ratings	Place_Name	Description	Category	City	Price	Rating	Time_Minutes	
0	1	179	3	Candi Ratu Boko	Situs Ratu Baka atau Candi Boko (Hanacaraka:ꦱꦺꦴꦫꦠꦸꦧꦺꦴꦏ)	Budaya	Yogyakarta	75000	4.6	90.0	{'lat': 1
1	1	344	2	Pantai Marina	Pantai Marina (bahasa Jawa: ꦥꦺꦴꦠꦶꦤꦠꦶꦩꦤꦶ)	Bahari	Semarang	3000	4.1	NaN	{'lat': -6.9
2	1	5	5	Atlantis Water Adventure	Atlantis Water Adventure atau dikenal dengan A...	Taman Hiburan	Jakarta	94000	4.5	60.0	{'lat': -6
3	1	373	3	Museum Kereta	Museum Kereta Api Ambarawa (bahasa	Budaya	Semarang	10000	4.5	60.0	-7.26459

Next steps: [Generate code with final_merged_df](#) [New interactive sheet](#)

```
final_merged_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 15 columns):
#   Column              Non-Null Count  Dtype
---  -
0   User_Id             10000 non-null  int64
1   Place_Id            10000 non-null  int64
2   Place_Ratings       10000 non-null  int64
3   Place_Name          10000 non-null  object
4   Description          10000 non-null  object
5   Category            10000 non-null  object
6   City                10000 non-null  object
7   Price               10000 non-null  int64
8   Rating              10000 non-null  float64
9   Time_Minutes        4628 non-null   float64
10  Coordinate           10000 non-null  object
11  Lat                  10000 non-null  float64
12  Long                 10000 non-null  float64
13  Location             10000 non-null  object
14  Age                  10000 non-null  int64
dtypes: float64(4), int64(5), object(6)
memory usage: 1.1+ MB
```

```
final_merged_df.columns

Index(['User_Id', 'Place_Id', 'Place_Ratings', 'Place_Name', 'Description',
      'Category', 'City', 'Price', 'Rating', 'Time_Minutes', 'Coordinate',
      'Lat', 'Long', 'Location', 'Age'],
      dtype='object')
```

```
# Check the final DataFrame
final_merged_df.head()
```

	User_Id	Place_Id	Place_Ratings	Place_Name	Description	Category	City	Price	Rating	Time_Minutes	
0	1	179	3	Candi Ratu Boko	Situs Ratu Baka atau Candi Boko (Hanacaraka:ꦱꦺꦴꦫꦠꦸꦧꦺꦴꦏ)	Budaya	Yogyakarta	75000	4.6	90.0	{'lat': 1
1	1	344	2	Pantai Marina	Pantai Marina (bahasa Jawa: ꦥꦺꦴꦠꦶꦤꦠꦶꦩꦤꦶ)	Bahari	Semarang	3000	4.1	NaN	{'lat': -6.9
2	1	5	5	Atlantis Water Adventure	Atlantis Water Adventure atau dikenal dengan A...	Taman Hiburan	Jakarta	94000	4.5	60.0	{'lat': -6
3	1	373	3	Museum Kereta	Museum Kereta Api Ambarawa (bahasa	Budaya	Semarang	10000	4.5	60.0	-7.26459

Next steps: [Generate code with final_merged_df](#) [New interactive sheet](#)

```
final_merged_df.isnull().sum()
```

```

0
User_Id      0
Place_Id     0
Place_Ratings 0
Place_Name   0
Description   0
Category     0
City         0
Price        0
Rating       0
Time_Minutes 5372
Coordinate    0
Lat          0
Long         0
Location     0
Age          0

```

dtype: int64

Only the Time_minutes columns has multiple Null values. We do not need that much this information probably.

```

df=final_merged_df.copy()
df

```

	User_Id	Place_Id	Place_Ratings	Place_Name	Description	Category	City	Price	Rating	Time_Minutes	
0	1	179	3	Candi Ratu Boko	Situs Ratu Baka atau Candi Boko (Hanacaraka:ꦏꦢꦶꦫꦠꦸꦧꦺꦴꦏ)	Budaya	Yogyakarta	75000	4.6	90.0	{
1	1	344	2	Pantai Marina	Pantai Marina (bahasa Jawa: ꦥꦤꦠꦶꦩꦤꦶ)	Bahari	Semarang	3000	4.1	NaN	{'lat':
2	1	5	5	Atlantis Water Adventure	Atlantis Water Adventure atau dikenal dengan A...	Taman Hiburan	Jakarta	94000	4.5	60.0	{'la
3	1	373	3	Museum Kereta Ambarawa	Museum Kereta Api Ambarawa (bahasa Inggris: In...	Budaya	Semarang	10000	4.5	60.0	-7.26
4	1	101	4	Kampung Wisata Sosro Menduran	Kampung wisata Sosromenduran merupakan kampung...	Budaya	Yogyakarta	0	4.0	NaN	-7.79
...	...	...	...	...	...	...	...	...	...	...	...
9995	300	425	2	Waterpark Kenjeran Surabaya	Waterpark Kenjeran Surabaya merupakan wisata k...	Taman Hiburan	Surabaya	35000	4.1	NaN	-7.24
9996	300	64	4	Museum Sasmita Loka Ahmad Yani	Museum Sasmita Loka Ahmad Yani adalah salah sa...	Budaya	Jakarta	2000	4.7	45.0	-6.20

Next steps: [Generate code with df](#) [New interactive sheet](#)

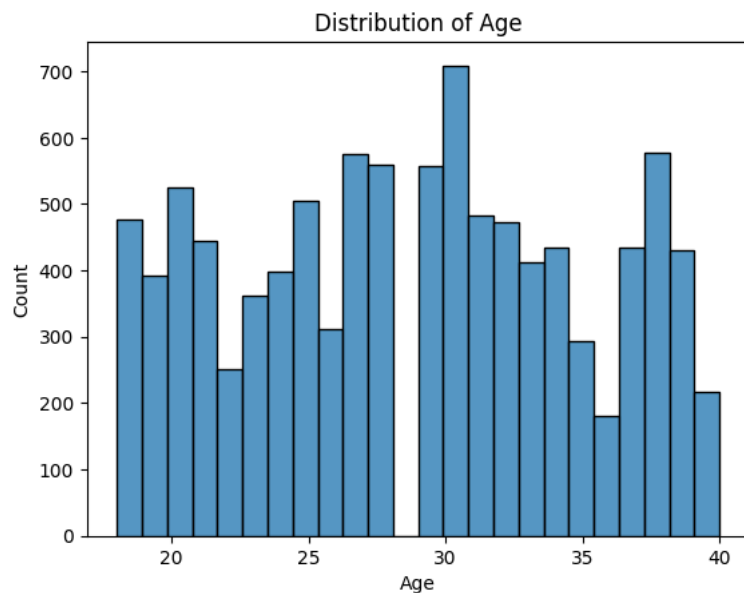
2. Exploring the data in depth

- Analyzing the age distribution of users visiting the places and rating them

```

# Age distribution of the user visited the places
sns.histplot(df['Age'])
plt.title('Distribution of Age')
plt.show()

```



- Identifying the places where most of these users (tourists) are coming from

```
# Group by 'Place_Name' and count unique 'User_Id'
tourist_count_by_place = df.groupby('Place_Name')['User_Id'].nunique().reset_index()

# Sort by the number of tourists (descending order)
tourist_count_by_place = tourist_count_by_place.sort_values(by='User_Id', ascending=False)

# Check the result
tourist_count_by_place.head()
```

	Place_Name	User_Id	
102	Gunung Lalakon	39	
272	Pantai Parangtritis	35	
86	Gereja Perawan Maria Tak Berdosa Surabaya	33	
95	Grand Maerakaca	33	
395	Taman Sungai Mudal	32	

Next steps: [Generate code with tourist_count_by_place](#) [New interactive sheet](#)

3. let's explore the locations and categories of tourist spots.

Different categories of tourist spots in the data

```
df['Category'].value_counts()
```

	count
Category	
Taman Hiburan	3053
Budaya	2683
Cagar Alam	2415
Bahari	1079
Pusat Perbelanjaan	385
Tempat Ibadah	385

dtype: int64

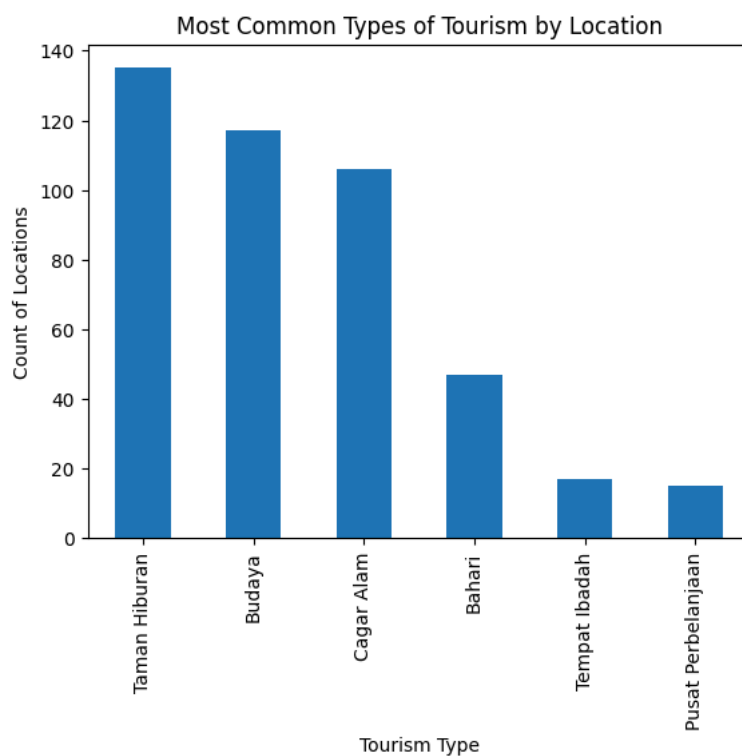
```
# Group by Place_Name and see the most frequent Category for each place
tourism_type_by_place = df.groupby('Place_Name')['Category'].apply(lambda x: x.mode()[0]).reset_index() # to get the most frequent category for each place
```

```
# Check the result
print(tourism_type_by_place.head(10))
```

	Place_Name	Category
0	Air Mancur Menari	Taman Hiburan
1	Air Terjun Kali Pancur	Cagar Alam
2	Air Terjun Kedung Pedut	Cagar Alam
3	Air Terjun Semirang	Cagar Alam
4	Air Terjun Sri Gethuk	Cagar Alam
5	Alive Museum Ancol	Taman Hiburan
6	Alun Alun Selatan Yogyakarta	Taman Hiburan
7	Alun-Alun Kota Bandung	Taman Hiburan
8	Alun-alun Utara Keraton Yogyakarta	Budaya
9	Amazing Art World	Budaya

```
# Plot of the most common tourism type for each place
import matplotlib.pyplot as plt
```

```
tourism_type_by_place['Category'].value_counts().plot(kind='bar')
plt.title('Most Common Types of Tourism by Location')
plt.xlabel('Tourism Type')
plt.ylabel('Count of Locations')
plt.show()
```



```
df['City'].value_counts()
```

	count
City	
Yogyakarta	2871
Bandung	2842
Jakarta	1920
Semarang	1317
Surabaya	1050

```
dtype: int64
```

```
# Example filter for nature-related categories
nature_tourism_df = df[df['Category'].str.contains('Cagar Alam|Nature|Eco', case=False, na=False)]
```

```
# Group by City and count the number of nature-related places
nature_places_by_city = nature_tourism_df.groupby('City')['Place_Name'].count().reset_index()

# Sort cities by the number of nature-related places
nature_places_by_city = nature_places_by_city.sort_values(by='Place_Name', ascending=False)
```


	Avg_rating
Place_Name	
Keraton Surabaya	3.967742
Puncak Gunung Api Purba - Nglanggeran	3.882353
Kampung Cina	3.842105
Teras Cikapundung BBWS	3.789474
Monumen Yogya Kembali	3.772727
Bukit Jamur	3.766667
Bukit Bintang Yogyakarta	3.764706
Monumen Nasional	3.722222
Glamping Lakeside Rancabali	3.714286
Pantai Baron	3.695652
Sanghyang Heuleut	3.655172
Stone Garden Citatah	3.647059
Masjid Agung Trans Studio Bandung	3.642857
Desa Wisata Gamplong	3.633333
Monumen Selamat Datang	3.631579
Dago Dreampark	3.619048
Wisata Agro Edukatif Istana Susu Cibugary	3.611111
Air Terjun Kedung Pedut	3.590909
Taman Pelangi	3.588235
Taman Keputran	3.576923
Rainbow Garden	3.565217
Obyek Wisata Goa Kreo	3.555556
Pintoe Langit Dahromo	3.555556
Tafso Barn	3.541667
Museum Tekstil	3.529412
Curug Batu Templek	3.523810
Taman Hiburan Rakyat	3.523810
The World Landmarks - Merapi Park Yogyakarta	3.523810
Bukit Moko	3.521739
Atlantis Water Adventure	3.520000

dtype: float64

```
# City loved based on the place ratings
df.groupby('City')['Place_Ratings'].mean().sort_values(ascending=False) # based on the Ratings by the users
```

	Place_Ratings
City	
Yogyakarta	3.104493
Bandung	3.078466
Surabaya	3.072381
Semarang	3.034169
Jakarta	3.010937

dtype: float64

```
# category of the places based on the place ratings
df.groupby('Category')['Place_Ratings'].mean().sort_values(ascending=False) # based on the Ratings by the users
```

Place_Ratings	
Category	
Taman Hiburan	3.117917
Cagar Alam	3.080745
Tempat Ibadah	3.080519
Budaya	3.034663
Bahari	3.006487
Pusat Perbelanjaan	2.945455

dtype: float64

df.head()

	User_Id	Place_Id	Place_Ratings	Place_Name	Description	Category	City	Price	Rating	Time_Minutes	
0	1	179	3	Candi Ratu Boko	Situs Ratu Baka atau Candi Boko (Hanacaraka:ꦑꦫꦠꦸꦧꦺꦴꦏ)	Budaya	Yogyakarta	75000	4.6	90.0	{'lat': 10.7, 'lng': 110.3}
1	1	344	2	Pantai Marina	Pantai Marina (bahasa Jawa: ꦑꦤꦠꦶꦩꦤꦶꦫ)	Bahari	Semarang	3000	4.1	NaN	{'lat': -6.9, 'lng': 110.4}
2	1	5	5	Atlantis Water Adventure	Atlantis Water Adventure atau dikenal dengan A...	Taman Hiburan	Jakarta	94000	4.5	60.0	{'lat': -6.1, 'lng': 106.8}
3	1	373	3	Museum Kereta	Museum Kereta Api Ambarawa (bahasa Jawa: ꦩꦸꦱꦺꦴꦩꦏꦼꦫꦠ)	Budaya	Semarang	10000	4.5	60.0	-7.26459

Next steps:

[Generate code with df](#)[New interactive sheet](#)Start coding or [generate](#) with AI.

5. Recommendation system Building

```
# Slicing the data from the whole data set
data = df[['User_Id', 'Place_Id', 'Place_Name', 'Place_Ratings']]
```

data.info() # check the information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ---
 0   User_Id         10000 non-null  int64
 1   Place_Id        10000 non-null  int64
 2   Place_Name      10000 non-null  object
 3   Place_Ratings   10000 non-null  int64
dtypes: int64(3), object(1)
memory usage: 312.6+ KB
```

```
# missing value counts in each column of the data
data.isnull().sum()
```

```
0
User_Id    0
Place_Id   0
Place_Name 0
Place_Ratings 0
```

dtype: int64

- There is no null values in any of the columns of the sliced data set

data.describe() # Overall description of the data

	User_Id	Place_Id	Place_Ratings
count	10000.000000	10000.000000	10000.000000
mean	151.292700	219.416400	3.066500
std	86.137374	126.228335	1.379952
min	1.000000	1.000000	1.000000
25%	77.000000	108.750000	2.000000
50%	151.000000	220.000000	3.000000
75%	226.000000	329.000000	4.000000
max	300.000000	437.000000	5.000000

```
len(data) # length of the data
```

```
10000
```

```
# Calculate the number of unique user IDs in the filtered ratings dataset
len(data['User_Id'].unique())
```

```
300
```

```
# Calculate the number of unique Place IDs in the dataset
len(data['Place_Id'].unique())
```

```
437
```

```
## calculation of Average number of ratings per users ##

import statistics

# Calculate the number of ratings per user
ratings_per_user = data.groupby('User_Id')['Place_Ratings'].count()

# Calculate the mean number of ratings per user
mean_ratings_per_user = statistics.mean(ratings_per_user.tolist())
mean_ratings_per_user

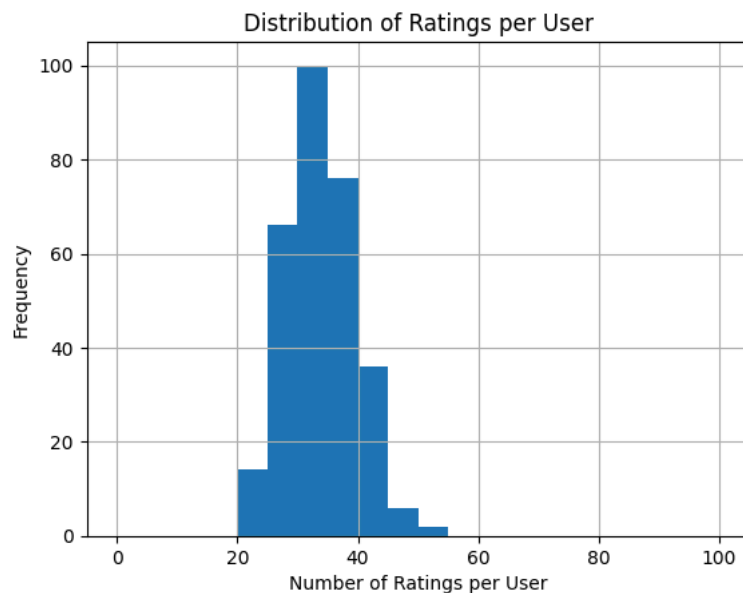
33.333333333333336
```

Observations:

- The average user has rated about 33 items, indicating a weakly active engagement in rating places within the dataset.
- With an average of 33 ratings per user, the dataset shows a imbalance of user activity that is skewed by extremely active users.

```
# visualization of per user ratings distribution
import matplotlib.pyplot as plt

# Plot a histogram of the number of ratings per user
ratings_per_user.hist(bins=20, range=(0,100))
plt.xlabel('Number of Ratings per User')
plt.ylabel('Frequency')
plt.title('Distribution of Ratings per User')
plt.show()
```

**Observations:**

- The distribution of ratings per user is right-skewed, with most users contributing a relatively small number of ratings, primarily below 60.
- The frequency decreases sharply as the number of ratings increases, and users with very high activity are extremely rare or absent in the dataset.

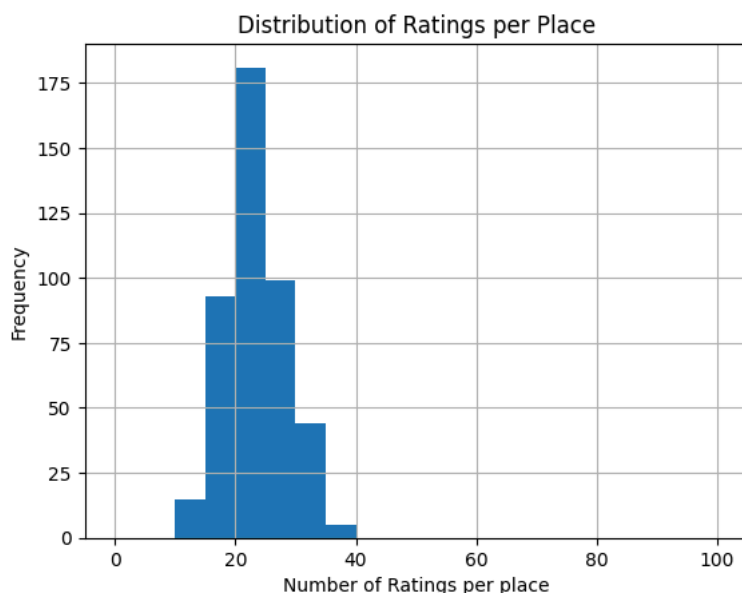
```
## calculation of Average number of ratings per place ##  
  
# Calculate the number of ratings per place  
ratings_per_place = data.groupby('Place_Id')['Place_Ratings'].count()  
  
# Calculate the mean number of ratings per place  
mean_ratings_per_place = statistics.mean(ratings_per_place.tolist())  
mean_ratings_per_place
```

22.88329519450801

Observations:

- On average, each place receives approximately 23 ratings, indicating a moderate level of user engagement per location.
- However, the mean alone does not capture the distribution of ratings across places, and further analysis is required to assess variability in engagement.

```
# visualization of per user ratings distribution  
  
# Plot a histogram of the number of ratings per place  
ratings_per_place.hist(bins=20, range=(0,100))  
plt.xlabel('Number of Ratings per place')  
plt.ylabel('Frequency')  
plt.title('Distribution of Ratings per Place')  
plt.show()
```



---Observations---

- The highest frequency of places falls within the range of approximately 10 to 40 ratings, as shown by the tallest bars in the histogram.
- Very few places receive more than 50 ratings, indicating a rapid decline in frequency as the number of ratings increases.
- The distribution is approximately symmetric, with most places receiving moderate engagement concentrated around 15–30 ratings. There is only a slight right skew, as a small number of places receive relatively higher ratings.

**** Filter place ratings greater than 30 to find the popular places with their ids**

```
# Filter the only places with 30 or more ratings
# threshold must be chosen based on the availability and need to check the sparsity in the data set after choosing the threshold
filtered_ratings_per_place = ratings_per_place[ratings_per_place >= 30]

# Extract the indices (place IDs) of popular places
popular_place = filtered_ratings_per_place.index.tolist()
len(popular_place)
```

49

**** Create user ratings per user greater than 35 to find the prolific users and their ids**

```
# Filter only users with 35 or more ratings
# threshold must be chosen based on the availability and need to check the sparsity in the data set after choosing the threshold
filtered_ratings_per_user_df = ratings_per_user[ratings_per_user >= 35]

# Extract the indices (user IDs) of prolific users
prolific_users = filtered_ratings_per_user_df.index.tolist()
len(prolific_users)
```

120

```
# Filter the ratings dataset to include only ratings for popular places
filtered_ratings_place = data[data.Place_Id.isin(popular_place)]
filtered_ratings_place

# Further filter the ratings dataset to include only ratings from prolific users
filtered_ratings_final = filtered_ratings_place[filtered_ratings_place.User_Id.isin(prolific_users)]
filtered_ratings_final

# Calculate the length of the filtered ratings dataset
len(filtered_ratings_final)
```

725

```
filtered_ratings_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 725 entries, 412 to 9969
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  -
 #   Column          Non-Null Count  Dtype
```

```

0  User_Id      725 non-null    int64
1  Place_Id     725 non-null    int64
2  Place_Name   725 non-null    object
3  Place_Ratings 725 non-null    int64
dtypes: int64(3), object(1)
memory usage: 28.3+ KB

```

```

# Create a pivot table to construct the user-place rating matrix
rating_matrix = filtered_ratings_final.pivot_table(index='User_Id', columns='Place_Id', values='Place_Ratings')

# Fill missing values (NaN) with 0
rating_matrix = rating_matrix.fillna(0)

# Display the first few rows of the user-place rating matrix
rating_matrix.head()

```

	Place_Id	8	29	43	53	55	56	78	88	89	114	...	355	366	367	368	369	377	402	416	430	437
User_Id																						
15		0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	0.0	5.0	0.0	4.0	0.0	0.0	0.0	0.0	0.0
17		0.0	0.0	0.0	0.0	0.0	0.0	4.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
19		0.0	0.0	0.0	0.0	0.0	3.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	4.0	0.0
26		0.0	0.0	0.0	5.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	3.0	0.0	0.0	0.0	0.0	0.0
27		0.0	0.0	0.0	0.0	0.0	0.0	3.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	4.0	0.0	0.0	0.0	0.0	0.0

5 rows × 49 columns

The **user-place rating matrix** indicates the interactions between users and places in the form of ratings. Each row represents a user, each column represents an Place id, and the values in the matrix represent the ratings given by users to the corresponding place id. This matrix provides a structured representation of user preferences and behaviors, serving as the foundation for collaborative filtering techniques to generate personalized recommendations.

- Define a function to find similar_users based on the Pierson correlation similarity of their rating vectors.

```

# Compute Pierson similarity
user_similarity = rating_matrix.T.corr()

```

```

def similar_users(user_id, similarity_matrix, k=5):

    if user_id not in similarity_matrix.index:
        return []

    similarities = similarity_matrix.loc[user_id].drop(user_id)

    # Fill NaN instead of dropping everything
    similarities = similarities.fillna(0)

    # Sort
    similarities = similarities.sort_values(ascending=False)

    return similarities.head(k).index.tolist()

```

```

current_user = 2

similar_user_indices = similar_users(current_user, user_similarity)

print(similar_user_indices)

[]

```

- Blank list means no strong similar users were found due to sparse or non-overlapping data. Lets chaeck for another user

```

current_user = 19

similar_user_indices = similar_users(current_user, user_similarity)

print(similar_user_indices)

[137, 143, 261, 58, 273]

```

- Check the sparcity in the data matrix

```
# Total cells
total_cells = rating_matrix.shape[0] * rating_matrix.shape[1]

# Non-zero (actual ratings)
non_zero = (rating_matrix != 0).sum().sum()

# Sparsity %
sparsity = 1 - (non_zero / total_cells)

print("Sparsity:", sparsity)
```

Sparsity: 0.8840136054421769

```
# Check the overlap between two users chosen in such a way that they are present in the rating matrix
user1 = rating_matrix.loc[19]
user2 = rating_matrix.loc[31]

# Find common rated items
common = ((user1 != 0) & (user2 != 0)).sum()

print("Common place between user 22 and 10:", common)
```

Common place between user 22 and 10: 0

```
# Check the overlap between two users
user1 = rating_matrix.loc[2]
user2 = rating_matrix.loc[31]

# Find common rated items
common = ((user1 != 0) & (user2 != 0)).sum()

print("Common items between user 22 and 10:", common)
```

```
-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.Int64HashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.Int64HashTable.get_item()

KeyError: 2
```

The above exception was the direct cause of the following exception:

```
-----
KeyError                                Traceback (most recent call last)
-----
      5 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812         raise KeyError(key) from err
    3813     except TypeError:
    3814         # If we have a listlike key, _check_indexing_error will raise

KeyError: 2
```

Next steps: [Explain error](#)

Pearson fails due to no overlap between these two users

```
# Check the similarity factors of user 19
sim = user_similarity.loc[19] # Check the pearson similarity of user 19
print(sim.head(20).sort_values(ascending=False))
```

```
User_Id
19    1.000000
58    0.388918
40    0.294442
45    0.171050
44    0.158698
55    0.073134
53    0.063299
27    0.027424
```



```

57    0.025341
28    0.000622
61   -0.073946
54   -0.092003
49   -0.107652
30   -0.119018
31   -0.132459
15   -0.138123
17   -0.139981
33   -0.169992
38   -0.172466
26   -0.183214
Name: 19, dtype: float64

```

similarity factors of user 19 with the other users. It looks user 58 has similar user behaviour

```

print(sim.describe()) # describes how many similar users are apresent with user 19

count    120.000000
mean      0.006807
std       0.173909
min      -0.190401
25%      -0.118549
50%      -0.052956
75%       0.078785
max       1.000000
Name: 19, dtype: float64

```

↳ Lets try for cosine similarities

- Define function to find similar_users based on the cosine similarity of their rating vectors.

```

from sklearn.metrics.pairwise import cosine_similarity
import operator

def similar_users(user_id, matrix, k=5):
    # Retrieve the user's vector from the rating matrix
    user = matrix[matrix.index == user_id]

    # Exclude the user's vector from the rating matrix
    other_users = matrix[matrix.index != user_id]

    # Calculate cosine similarity between the user and other users
    similarities = cosine_similarity(user, other_users)[0].tolist()

```